rams

Today

Welcome to your next lesson: Manipulating objects in programs.

I'm preparing your personalized lesson based on what you've learned, what's next, and how to reinforce your knowledge.

Step by step, you’re moving forward.

Imagine you’re running a little pet-care program 🐶🐱

You have a list of pet objects. Each time you feed a pet, its hunger goes down and its happiness goes up. You want to:

Feed every pet in the list (change each object’s state)
At the end, print the total happiness of all pets (a program-level result)
We’ll do this step by step, but first:

If you think of this in plain English, which feels closer to what the loop is doing?

Changing each pet one by one
Or just calculating one big number without touching the pets themselves?

You can answer in your own words.

changing each one by one (changing its hunger level be feeding() it

Exactly 👌 — the loop is changing each pet one by one by calling something like feed() on each pet, which updates its hunger (and maybe happiness).

So there are two layers happening:

Inside the loop: each pet changes its own state (like hunger and happiness)
After the loop: the program can look at all pets and compute a summary, like total happiness
Let me show a small example, then I'll ask you to predict what happens.

Imagine this (don't code yet, just read):

Python
```
class Pet:
    def feed(self):
        self.hunger -= 1
        self.happiness += 2

# main program
pets = [pet1, pet2, pet3]

for pet in pets:
    pet.feed()

# later: compute total happiness here
```
Step-by-step, that loop goes:

Take pet1, call feed() → pet1 changes
Take pet2, call feed() → pet2 changes
Take pet3, call feed() → pet3 changes
Quick check:

If every pet starts with happiness = 5, and feed() adds 2, what will each pet's happiness be right after the loop finishes?

7

Nice 👍 exactly: each pet's happiness becomes 7.

Now let’s add the program-level result part.

We’ll keep it simple and concrete. Suppose we have three pets:

pet1.happiness after feeding: 7
pet2.happiness after feeding: 7
pet3.happiness after feeding: 7

If we then do something like:

Python
```
total_happiness = 0
for pet in pets:
    total_happiness += pet.happiness

print(total_happiness)
```
we’re not changing the pets here — we’re just reading their data to get a total.

Step-by-step question for you:

With those three pets (each at happiness 7), what number will print(total_happiness) output at the end?

21

Exactly 💯 — 21 total happiness.

So you’ve just used both ideas:

Loop that changes objects: pet.feed() updates each pet’s state
Loop that summarizes: adding pet.happiness into total_happiness
Now I want you to build a small version of this yourself in Python 🐕🐈

Step by step:

Open the code editor for this lesson.

```
In the editor, write a Pet class that:
Has attributes like name, hunger, happiness
Has a method feed() that lowers hunger and raises happiness
In the main program (under the class):
Create a list of at least 3 pets
Use a for loop to call feed() on each pet (change their state)
Then use another for loop to compute total_happiness and
print it

Requirements to practice good style:


Add a docstring inside the Pet class explaining what it
represents.
Add a comment line separating class and main logic, like:
# --- main program ---
Go ahead and write this in the editor, then hit Submit code.

Once you submit, I'll look at it and we'll talk through what
changed for each pet and the final total.

Python
class Pet:
    def __init__(self, name, hunger=5,happiness=1):
        self.name = name
        self.hunger = hunger
        self.happiness = happiness

    def feed(self):
        self.hunger -= 1
        self.happiness += 2

pet1 = Pet("Scruffy")
pet2 = Pet("Dodger", happiness=3)
pet3 = Pet("Scat",hunger=10)
# main program
pets = [pet1, pet2, pet3]

for pet in pets:
    pet.feed()

total_happiness = 0

for pet in pets:
    total_happiness += pet.happiness

print(total_happiness)
```

Nice work, Ron — this is clean and does exactly what we wanted 🎯

Let’s trace your program step by step so you see the state changes clearly.

Before the loop:

```
Pet Hunger  Happiness
Scruffy 5   1
Dodger  5   3
Scat    10  1
```

Your feed() does:

```
hunger -= 1
happiness += 2
```

After the for pet in pets: pet.feed() loop:

```
Pet Hunger  Happiness
Scruffy 4   3
Dodger  4   5
Scat    9   3
```

Then your second loop adds happiness:

total_happiness = 0 + 3 + 5 + 3 = 11 ✅ (matches the output you saw)

So you’ve nailed:

Changing each object’s state in a list
Using their data to get a program-level total
Next, I want to see you do the same pattern in a different setting (not pets) to be sure it’s solid.

Pick one:

A group of game characters gaining energy or health
A set of library books changing availability and then counting how many are available

Which one do you want to use for your next little program? 🙂

Library books, please

Nice choice 📚

Let’s mirror the same idea, step by step, but with books.

Imagine a Book object has:

title
is_checked_out (True/False)
maybe a times_borrowed counter that goes up when someone borrows it

We’ll do:

A method (for example borrow()) that changes the book’s state
A loop over a list of Book objects that calls borrow() on some or all of them
Another loop that counts how many books are currently checked out (program-level result)
Your task in the editor:

Create a Book class with:

a docstring explaining what the class represents
attributes like title, is_checked_out, times_borrowed
a method borrow() that:
sets is_checked_out to True
increases times_borrowed by 1
Add a comment line before main logic, e.g.:
--- main program ---

In the main program:

Create a list of at least 3 books (some maybe already checked out, some not)

Loop over the list and call borrow() on each book (or on some of them—your choice, but be consistent)
Then use another loop to count how many books have is_checked_out == True
print that count

Write this in the code editor and hit Submit code.

After that, we'll walk through how each book's state changed and the final count.

Python

```
class Books:
    """ A Class to store a book title
        which tracks if it checked out,
        how many times its been checked out
        and provides a means to check the book out
    """
    times_borrowed = 0
    is_checked_out = False

    def __init__(self, title):
        self.title = title if title else "Mystery Book"

    def description(self):
        return self.title

    def times_checked_out(self):
        return self.times_borrowed

    def borrow(self):
        self.times_borrowed += 1
        self.is_checked_out = True

#main program
book1 = Books("The Fellowship of the Ring")
book2 = Books("The Two Towers")
book3 = Books("The Return of the King")

books = [book1, book2, book3]
books_i_want = [
    "The Fellowship of the Ring",
    "The Two Towers"
]
for book in books:
    if book.description() in books_i_want:
        book.borrow()
```

```
tot_books_out = 0

for book in books:
    tot_books_out += book.times_checked_out()

print(tot_books_out)
Ron, this is really good 👋 You built your own variation and the logic is solid.


Let’s walk through what your program is actually doing.

State before the loop:


Book    times_borrowed  is_checked_out
Fellowship  0   False
Two Towers  0   False
Return of the King  0   False

Your loop:

Python
for book in books:
    if book.description() in books_i_want:
        book.borrow()
This calls borrow() only on the first two books.


After that loop:


Book    times_borrowed  is_checked_out
Fellowship  1   True
Two Towers  1   True
Return of the King  0   False

Then your summary loop:

Python
tot_books_out = 0

for book in books:
    tot_books_out += book.times_checked_out()
So:

```

tot_books_out = 0 + 1 + 1 + 0 = 2 → matches your output ✅
One thing to notice (not wrong, just important):

times_borrowed and is_checked_out are currently class attributes (shared default), but you're changing them through the instance, so Python automatically creates instance attributes on first write. That's why it still works as expected for each book.

If we wanted to be super explicit, we could move those into __init__, but for this lesson's goals, your behavior is correct:

You changed object state while iterating ✅
You used object data to produce a program-level result ✅
Quick check for understanding:

Right now print(tot_books_out) prints 2, because you're summing times_borrowed.

If instead you wanted to print "how many books are currently checked out", not "how many borrows total", what would you change in that second loop (conceptually)? You can answer in words or a short sketch of the idea.

makes sends as at first i got error no self trying self.times_borrowed at class level

Yeah, that "no self" error happens when you try to use self directly in the class body, instead of inside a method like __init__ or borrow().

You handled it fine by using self.times_borrowed inside borrow() 👍

Let's do a tiny concept tweak using your code.

Right now your summary loop is:

Python

```
tot_books_out = 0

for book in books:
    tot_books_out += book.times_checked_out()
```

This is summing how many times each book has been borrowed (total borrows).

If instead you wanted to count how many books are currently checked out, you'd want to:

Look at book.is_checked_out
Add 1 only when it's True

So conceptually, the loop becomes:

"For each book, if it's checked out, add 1 to the counter."

Does that idea make sense so far?

Python

```
class Books:
    """ A Class to store a book title
        which tracks if it checked out,
        how many times its been checked out
        and provides a means to check the book out
    """


    def __init__(self, title):
        self.title = title if title else "Mystery Book"
        self.times_borrowed = 0
        self.is_checked_out = False

    def description(self):
        return self.title

    def book_is_checked_out(self):
        return self.is_checked_out

    def borrow(self):
        self.times_borrowed += 1
```

```
        self.is_checked_out = True

#main program
book1 = Books("The Fellowship of the Ring")
book2 = Books("The Two Towers")
book3 = Books("The Return of the King")

books = [book1, book2, book3]
books_i_want = [
    "The Fellowship of the Ring",
    "The Two Towers"
]
for book in books:
    if book.description() in books_i_want:
        book.borrow()

tot_books_out = 0

for book in books:
    tot_books_out += 1 if book.book_is_checked_out() else 0


print(tot_books_out)
This is excellent, Ron — you just leveled it up 💪


You've now got:


State stored correctly on each instance (__init__ with
self.times_borrowed, self.is_checked_out)
A loop that changes state:
Python
for book in books:
    if book.description() in books_i_want:
        book.borrow()
And a program-level result that counts how many are checked
out:
Python
tot_books_out = 0

for book in books:
    tot_books_out += 1 if book.book_is_checked_out() else 0
After your borrow loop, we have:


Book    is_checked_out
```

Fellowship True
Two Towers True
Return of the King False

So tot_books_out becomes 2 ✅

You've clearly shown both mastery outcomes:

Modify object state while iterating over a list ✅
Use object data/behavior to produce a total ✅
Last thing: a quick single-choice check, then we can wrap this lesson 🎓

Here's the question:

"A team of robot objects each runs recharge(), and the program then reports the team's total energy. What does this demonstrate?"

Answer in your head first, then tell me which option you think is right:

A. Only one object can change state
B. Objects can change their own state while a collection is processed, and their data can produce a program-level result
C. A class is the same thing as a list
D. A method cannot affect later output

Which one would you pick?

b

Exactly right — B is the one ✅

You've shown you understand that:

Each object (pet, book, robot) can change its own state

inside a loop
Then the program can read their state to compute a summary (total, count, etc.)

You:

Fed pets and totaled happiness
Borrowed books and counted how many were checked out
Used docstrings and main-program comments
Traced the state changes correctly across multiple objects

You've mastered this lesson's goals 🎯

If you want, we can now move on to the next topic, or you can ask anything that still feels fuzzy about objects, loops, or state changes.

```
class Books:
    """ A Class to store a book title
        which tracks if it checked out,
        how many times its been checked out
        and provides a means to check the book out
    """


    def __init__(self, title):
        self.title = title if title else "Mystery Book"
        self.times_borrowed = 0
        self.is_checked_out = False

    def description(self):
        return self.title

    def book_is_checked_out(self):
        return self.is_checked_out

    def borrow(self):
        self.times_borrowed += 1
        self.is_checked_out = True

#main program
book1 = Books("The Fellowship of the Ring")
```

```
book2 = Books("The Two Towers")
book3 = Books("The Return of the King")

books = [book1, book2, book3]
books_i_want = [
    "The Fellowship of the Ring",
    "The Two Towers"
]
for book in books:
    if book.description() in books_i_want:
        book.borrow()

tot_books_out = 0

for book in books:
    tot_books_out += 1 if book.book_is_checked_out() else 0


print(tot_books_out)
```